

Prepared by : **Shiva Prasaath SB**
To : CAMS

Problem Statement:

Enterprise finance teams manually review hundreds of vendor invoices monthly — a process that is slow, error-prone, and lacks auditability. A single missed rate violation or malformed invoice can result in overpayments, compliance breaches, or fraudulent billing going undetected.

CamsAI Invoice Auditor is a fully automated, multi-agent AI system that ingests vendor invoices at scale, validates them against approved contract thresholds using vector similarity search, flags anomalies in real time, and routes compliant invoices directly to the financial ledger — all without human intervention.

The system handles three critical failure modes:

- **Rate deviation** — line items exceeding contracted unit rates
- **Structural corruption** — malformed payloads with invalid data
- **Limit breaches** — invoices exceeding approved billing totals

Every decision is logged to a persistent audit trail, anomalies are escalated to Slack instantly, and the entire pipeline runs as a self-contained Docker stack deployable with a single command.

Solution Architecture:

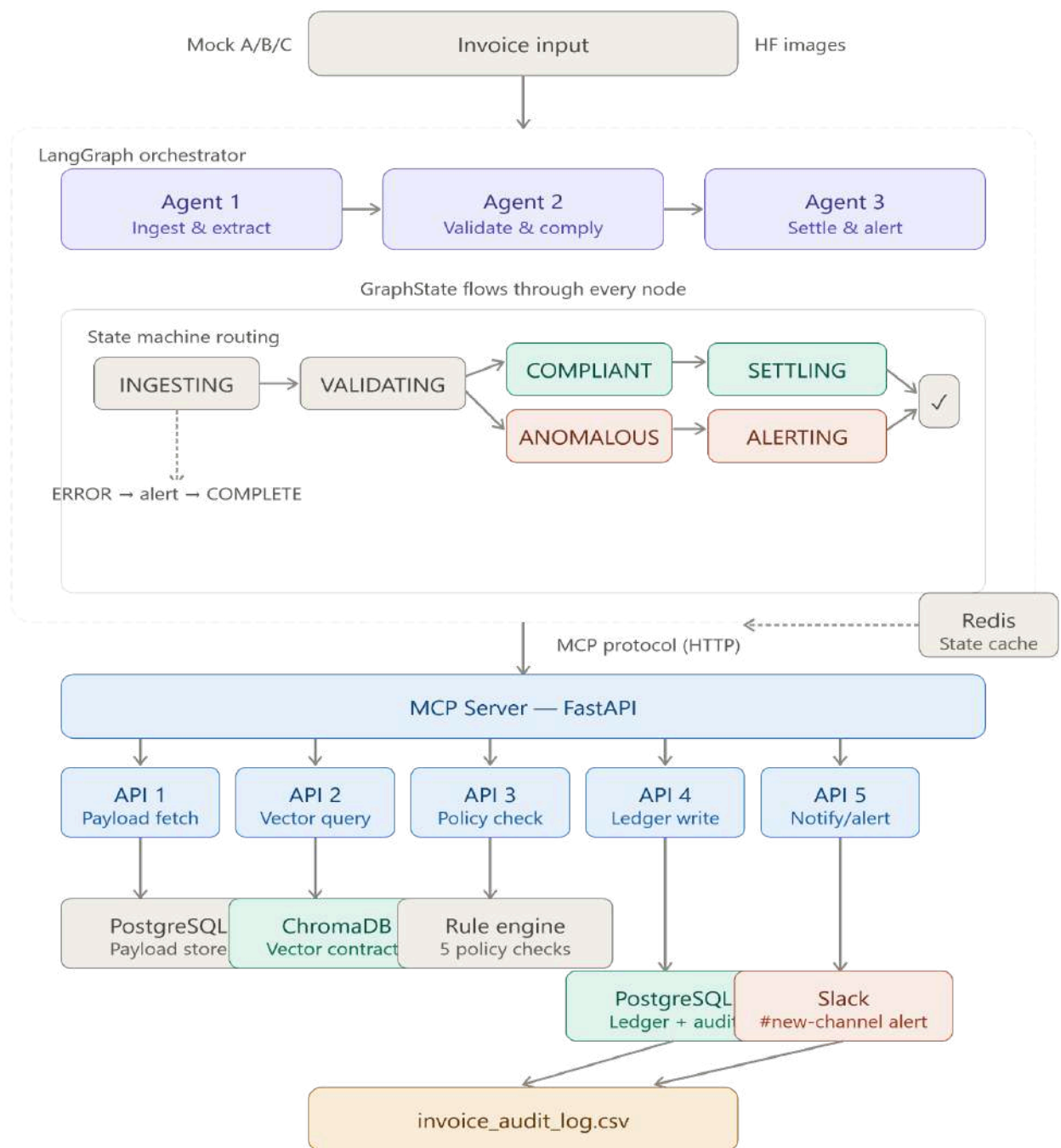
A three-agent LangGraph orchestration system where each agent communicates exclusively through a central MCP Server exposing 5 REST APIs. No agent talks directly to any database or external service — all interactions are mediated through the MCP layer.

Layer 1 — Orchestration (LangGraph)

The core is a compiled LangGraph **StateGraph** that manages a shared **GraphState** object across all agents. Each node is an async Python function. Edges are conditional — the next node is determined by the current state value after each agent completes.

```
START → INGEST → VALIDATE → SETTLE → END
                        ↘ ALERT → END
                        ↘ ERROR → ALERT → END
```

State transitions are deterministic and logged at every step. Retry logic handles transient network failures with exponential backoff up to 3 attempts.



Layer 2 — Agents

Agent 1: Ingestion & Extraction

Two ingestion modes selected automatically by invoice number prefix:

- **INV-*** → fetches pre-structured JSON from MCP API 1
- **HF:dataset:index** → downloads invoice image from HuggingFace (Voxel51 or mychen76 dataset), encodes to base64, sends to Claude Sonnet claude-sonnet-4-6 Vision API, parses returned JSON into typed **InvoicePayload**

All parsed data is validated through Pydantic v2 before proceeding. Empty line items get a placeholder so malformed invoices flow through the pipeline rather than crashing it.

Agent 2: Validation & Compliance

Two-step validation:

1. Queries ChromaDB via MCP API 2 using cosine similarity search against vendor contract embeddings. Returns contract-specific thresholds (max unit rate, max total) for the vendor.
2. Sends invoice + contract context to MCP API 3 (deterministic rule engine). Rules checked: unit rate per line item, invoice total limit, line item sum mismatch, negative values, empty items. Returns compliance score (0–100) and anomaly flags.

Agent 3: Reporting & Settlement

Branches based on validation result:

- **COMPLIANT** → writes full invoice + line items to PostgreSQL via MCP API 4. Uses **RETURNING id** to prevent foreign key violations on re-runs.
- **ANOMALOUS / ERROR** → calls MCP API 5 (notification), then fires Slack webhook with structured block message including vendor, amount, severity, and full anomaly detail.

Every run appends a row to **invoice_audit_log.csv** immediately after completion.

Layer 3 — MCP Server (FastAPI)

API	Method	Endpoint	Function
API 1	GET	/api/v1/payloads/{id}	Returns invoice payload. Persists to PostgreSQL on seed so payloads survive server restarts.
API 2	POST	/api/v1/vector/query	Queries ChromaDB vendor_contracts collection using cosine similarity. Returns top matching contracts with metadata.
API 3	POST	/api/v1/policy/check	Stateless rule engine. Applies 5 rules, returns compliance score, flags, violations, recommendation.
API 4	POST	/api/v1/ledger/write	Upserts invoice to PostgreSQL with ON CONFLICT. Writes line items and audit event atomically.
API 5	POST	/api/v1/notify/alert	Sends structured Slack alert. Falls back to log-only if SMTP not configured.

Layer 4 — Data Stores

PostgreSQL 15 - Five tables: `invoices`, `line_items`, `contracts`, `audit_events`, `invoice_payloads`. Schema created automatically on first boot via `init_db.sql`.

ChromaDB 0.5.3 - Single collection `vendor_contracts` with cosine similarity index. Seeded with 3 vendor contracts on startup. Used by Agent 2 for contract term retrieval.

Redis 7 - Key-value store for orchestrator state persistence. Each completed invoice writes final `GraphState` as JSON with 1-hour TTL. Key format: `invoice:state:{invoice_number}`.

Layer 5 — External Integrations

HuggingFace Datasets

- `Voxel51/high-quality-invoice-images-for-ocr` — 8,181 synthetic JPEG invoices. Ran 93 samples in this assessment.
- `mychen76/invoices-and-receipts_ocr_v1` — 1,489 real invoices with ground truth JSON. Ran 100 samples in this assessment.
- Total: 193 real invoice images processed end-to-end through the full pipeline.
- Accessed via `datasets` Python library with streaming mode (no full download required).

Anthropic Claude claude-sonnet-4-6 - Used exclusively by Agent 1 for Vision OCR. Receives base64-encoded invoice image, returns structured JSON with all invoice fields extracted (invoice number, vendor, line items, totals, dates).

Slack - Incoming webhook to `#new-channel` in workspace `shivaprasaath`. Fires on every ANOMALOUS or ERROR invoice with rich block message format including vendor name, total amount, severity, and full anomaly flag details.

Deployment

Single `docker-compose.yml` defines 6 services with health checks and strict dependency ordering:

```
postgres + redis + chromadb (parallel)
```

```
↓
```

```
mcp_server
```

```
↓
```

```
seeder (runs once, exits)
```

```
↓
```

orchestrator (runs once, exits)

Single command deployment:

docker-compose up --build

Output Images

Orchestrator running :

```
MINGW64/C:\Cams_Test\compose-invoice-auditor
invoice_orchestrator 2026-07-11 17:28:06.135 [orchestrator] INFO - [COMPLETE: INV-SCENARIO-B
invoice_orchestrator 2026-07-11 17:28:06.135 [orchestrator] INFO - Final State : OrchestratorState.COMPLETE
invoice_orchestrator 2026-07-11 17:28:06.135 [orchestrator] ERROR - Validate Err: API 3 (policy) connection error:
invoice_orchestrator 2026-07-11 17:28:06.135 [orchestrator] INFO -
invoice_orchestrator 2026-07-11 17:28:06.643 [orchestrator] INFO -
invoice_orchestrator 2026-07-11 17:28:06.644 [orchestrator] INFO - SCENARIO : Structural Crash - error capture, alert
invoice_orchestrator 2026-07-11 17:28:06.644 [orchestrator] INFO - Invoice : INV-SCENARIO-C
invoice_orchestrator 2026-07-11 17:28:06.644 [orchestrator] INFO -
invoice_orchestrator 2026-07-11 17:28:06.644 [orchestrator] INFO -
invoice_orchestrator 2026-07-11 17:28:06.648 [orchestrator] INFO - [START: INV-SCENARIO-C
invoice_orchestrator 2026-07-11 17:28:06.659 [orchestrator] INFO - [Orchestrator] ▶ INGEST | invoice=INV-SCENARIO-C
invoice_orchestrator 2026-07-11 17:28:06.659 [Agent1-Ingestion] INFO - [Agent1] Structured path: Invoice=INV-SCENARIO-C
invoice_orchestrator 2026-07-11 17:28:21.697 [Agent1-Ingestion] ERROR - [Agent1] API 1 connection error:
invoice_orchestrator 2026-07-11 17:28:21.698 [orchestrator] INFO - [Orchestrator] ▶ INGEST done | state=OrchestratorState.ERROR
invoice_orchestrator 2026-07-11 17:28:21.703 [orchestrator] WARNING - [Orchestrator] Transient error - retry 1/3
invoice_orchestrator 2026-07-11 17:28:21.707 [orchestrator] INFO - [Orchestrator] ▶ VALIDATE
invoice_orchestrator 2026-07-11 17:28:21.708 [orchestrator] INFO - [Orchestrator] ▶ VALIDATE done | state=OrchestratorState.ERROR
invoice_orchestrator 2026-07-11 17:28:21.720 [orchestrator] INFO - [Orchestrator] ▶ ALERT
invoice_orchestrator 2026-07-11 17:28:21.720 [Agent1-Settlement] WARNING - [Agent3] Sending alert for INV-SCENARIO-C - severity: critical - flags: 1
invoice_orchestrator 2026-07-11 17:28:39.039 [Agent1-Settlement] ERROR - [Agent3] API 3 (notify) connection error:
invoice_orchestrator 2026-07-11 17:28:39.040 [orchestrator] INFO - [Orchestrator] ▶ ALERT done | state=OrchestratorState.COMPLETE
invoice_orchestrator 2026-07-11 17:28:39.044 [orchestrator] INFO -
invoice_orchestrator 2026-07-11 17:28:39.044 [orchestrator] INFO - [COMPLETE: INV-SCENARIO-C
invoice_orchestrator 2026-07-11 17:28:39.044 [orchestrator] INFO - Final State : OrchestratorState.COMPLETE
invoice_orchestrator 2026-07-11 17:28:39.044 [orchestrator] ERROR - Extract Err : API 1 connection error:
invoice_orchestrator 2026-07-11 17:28:39.044 [orchestrator] INFO - Validate Err: No extracted invoice in state - Agent 1 may have failed
invoice_orchestrator 2026-07-11 17:28:39.044 [orchestrator] INFO -
invoice_orchestrator 2026-07-11 17:28:39.551 [orchestrator] INFO -
invoice_orchestrator 2026-07-11 17:28:39.551 [orchestrator] INFO - SCENARIO : OCR - VoxelSI sample #0 (tabular layout, currency parsing)
invoice_orchestrator 2026-07-11 17:28:39.551 [orchestrator] INFO - Invoice : HF:voxelSI:0
invoice_orchestrator 2026-07-11 17:28:39.551 [orchestrator] INFO -
invoice_orchestrator 2026-07-11 17:28:39.555 [orchestrator] INFO -
invoice_orchestrator 2026-07-11 17:28:39.555 [orchestrator] INFO - [START: HF:voxelSI:0
invoice_orchestrator 2026-07-11 17:28:39.555 [orchestrator] INFO -
invoice_orchestrator 2026-07-11 17:28:39.567 [orchestrator] INFO - [Orchestrator] ▶ INGEST | invoice=HF:voxelSI:0
invoice_orchestrator 2026-07-11 17:28:39.568 [Agent1-Ingestion] INFO - [Agent1] OCR path: dataset=voxelSI sample=0
invoice_orchestrator 2026-07-11 17:28:39.588 [Agent1-OCR] INFO - [Agent1-OCR] Loading voxelSI sample #0
invoice_orchestrator 2026-07-11 17:28:43.098 [https] INFO - HTTP Request: GET https://datasets-server.huggingface.co/rows?dataset=voxelSI/high-quality-invoice-images-for-ocr&split=train&offset=0&length=1 "HT
P/1.1 422 Unprocessable Content"
invoice_orchestrator 2026-07-11 17:28:43.100 [Agent1-Ingestion] WARNING - [Agent1] HF Dataset API error (voxelSI[0]): Client error '422 Unprocessable Content' for url 'https://datasets-server.huggingface.co/rows?dataset=voxelSI/high-quality-invoice-images-for-ocr&split=train&offset=0&length=1'
invoice_orchestrator 2026-07-11 17:28:43.101 [Agent1-Ingestion] ERROR - [Agent1] HF OCR ingestion failed: Could not fetch image from voxelSI dataset (sample #0). Ensure HF_TOKEN is set if the dataset require authentication.
invoice_orchestrator 2026-07-11 17:28:43.101 [orchestrator] INFO - [Orchestrator] ▶ INGEST done | state=OrchestratorState.ERROR
invoice_orchestrator 2026-07-11 17:28:43.111 [Agent1-Settlement] WARNING - [Agent3] Sending alert for HF:voxelSI:0 - severity: critical - flags: 1
invoice_orchestrator 2026-07-11 17:28:43.111 [orchestrator] INFO - [Orchestrator] ▶ ALERT
```

Sample Data Output

```
MINGW64 ~/c/Cams_Test_F/camsai-invoice-auditor
invoice_orchestrator 2026-07-11 07:22:43,465 [orchestrator] INFO -
invoice_orchestrator 2026-07-11 07:22:43,465 [orchestrator] INFO - SCENARIO : Structural Crash - error capture, alert
invoice_orchestrator 2026-07-11 07:22:43,465 [orchestrator] INFO - Invoice : INV-SCENARIO-C
invoice_orchestrator 2026-07-11 07:22:43,470 [orchestrator] INFO -
invoice_orchestrator 2026-07-11 07:22:43,471 [orchestrator] INFO - # START: INV-SCENARIO-C
invoice_orchestrator 2026-07-11 07:22:43,471 [orchestrator] INFO -
invoice_orchestrator 2026-07-11 07:22:43,484 [orchestrator] INFO - [orchestrator] * INGEST | invoice-INV-SCENARIO-C
invoice_orchestrator 2026-07-11 07:22:43,484 [Agent1-Ingestion] INFO - [Agent1] Structured path: Invoice-INV-SCENARIO-C
invoice_orchestrator 2026-07-11 07:22:43,547 [http] INFO - HTTP Request: GET http://mcp_server:8080/api/v1/payloads/INV-SCENARIO-C "HTTP/1.1 404 Not Found"
invoice_orchestrator 2026-07-11 07:22:43,548 [Agent1-Ingestion] ERROR - [Agent1] Invoice INV-SCENARIO-C not found
invoice_orchestrator 2026-07-11 07:22:43,548 [orchestrator] INFO - [orchestrator] * INGEST done | state=OrchestratorState.ERROR
invoice_orchestrator 2026-07-11 07:22:43,567 [orchestrator] INFO - [orchestrator] * ALERT
invoice_orchestrator 2026-07-11 07:22:43,567 [Agent1-Settlement] WARNING - [Agent1] Sending alert for INV-SCENARIO-C - severity: critical - Flags: 1
invoice_orchestrator 2026-07-11 07:22:43,610 [http] INFO - HTTP Request: POST http://mcp_server:8080/api/v1/notify/alert "HTTP/1.1 200 OK"
invoice_orchestrator 2026-07-11 07:22:43,612 [Agent1-Settlement] INFO - [Agent1] - Alert sent via log: msg_id=alert-c5388c9c2e9
invoice_orchestrator 2026-07-11 07:22:43,613 [orchestrator] INFO - [orchestrator] * ALERT done | state=OrchestratorState.COMPLETE
invoice_orchestrator 2026-07-11 07:22:43,618 [orchestrator] INFO -
invoice_orchestrator 2026-07-11 07:22:43,618 [orchestrator] INFO - @ COMPLETE: INV-SCENARIO-C
invoice_orchestrator 2026-07-11 07:22:43,619 [orchestrator] INFO - Final State : OrchestratorState.COMPLETE
invoice_orchestrator 2026-07-11 07:22:43,619 [orchestrator] INFO - Alert sent : log (alert-c5388c9c2e9)
invoice_orchestrator 2026-07-11 07:22:43,619 [orchestrator] ERROR - Extract Err : Invoice INV-SCENARIO-C not found
invoice_orchestrator 2026-07-11 07:22:43,619 [orchestrator] INFO -
invoice_orchestrator 2026-07-11 07:22:44,128 [orchestrator] INFO -
invoice_orchestrator 2026-07-11 07:22:44,128 [orchestrator] INFO - FINAL RESULTS SUMMARY
invoice_orchestrator 2026-07-11 07:22:44,128 [orchestrator] INFO - [OrchestratorState.COMPLETE] INV-SCENARIO-A
invoice_orchestrator 2026-07-11 07:22:44,128 [orchestrator] INFO - Fully Compliant - route to Ledger
invoice_orchestrator 2026-07-11 07:22:44,129 [orchestrator] INFO - Compliance score: 100.0/100
invoice_orchestrator 2026-07-11 07:22:44,130 [orchestrator] INFO - [OrchestratorState.COMPLETE] INV-SCENARIO-B
invoice_orchestrator 2026-07-11 07:22:44,132 [orchestrator] INFO - Anomalous Rate Deviation - skip Ledger, alert
invoice_orchestrator 2026-07-11 07:22:44,133 [orchestrator] INFO - [OrchestratorState.COMPLETE] INV-SCENARIO-C
invoice_orchestrator 2026-07-11 07:22:44,133 [orchestrator] INFO - Structural Crash - error capture, alert
invoice_orchestrator 2026-07-11 07:22:44,135 [orchestrator] INFO -
invoice_orchestrator 2026-07-11 07:22:44,135 [orchestrator] INFO - To run ALL images from each dataset (not just 10 samples),
invoice_orchestrator 2026-07-11 07:22:44,136 [orchestrator] INFO - see the comment block in agents/orchestrator.py main() -
invoice_orchestrator 2026-07-11 07:22:44,136 [orchestrator] INFO - change range(5) to range(N) where N is the number you want,
invoice_orchestrator 2026-07-11 07:22:44,139 [orchestrator] INFO - or set VOXEL51_SAMPLES and MYCHEN76_SAMPLES env vars.
invoice_orchestrator 2026-07-11 07:22:44,139 [orchestrator] INFO -

MINGW64 ~/c/Cams_Test_F/camsai-invoice-auditor
docker exec -it invoice_ledger_db psql -u auditor -d invoice_ledger -c "SELECT invoice_number, vendor_name, total_amount, status, compliance_score FROM Invoices;"
invoice_number | vendor_name | total_amount | status | compliance_score
-----
INV-SCENARIO-A | Acme Cloud Services | 4000.00 | approved | 100.00
(1 row)

What's next:
Try Docker Debug for seamless, persistent debugging tools in any container or image = docker debug invoice_ledger_db
Learn more at https://docs.docker.com/go/debug-cli/
```

Whole dataset from hugging face calling api's from Anthropic

```
continue
key = row[1] if len(row) > 1 else ''
if not key or key in seen:
    continue
seen.add(key)

decision = row[6] if len(row) > 6 else ''
if 'APPROVED' in decision:
    approved += 1
elif 'FLAGGED' in decision:
    flagged += 1
elif 'ERROR' in decision:
    error += 1

total = approved + flagged + error
print(f'Unique Invoices : {total}')
print(f'Approved : {approved}')
print(f'Flagged : {flagged}')
print(f'Error : {error}')

Unique Invoices : 28
Approved : 18
Flagged : 10
Error : 0
```

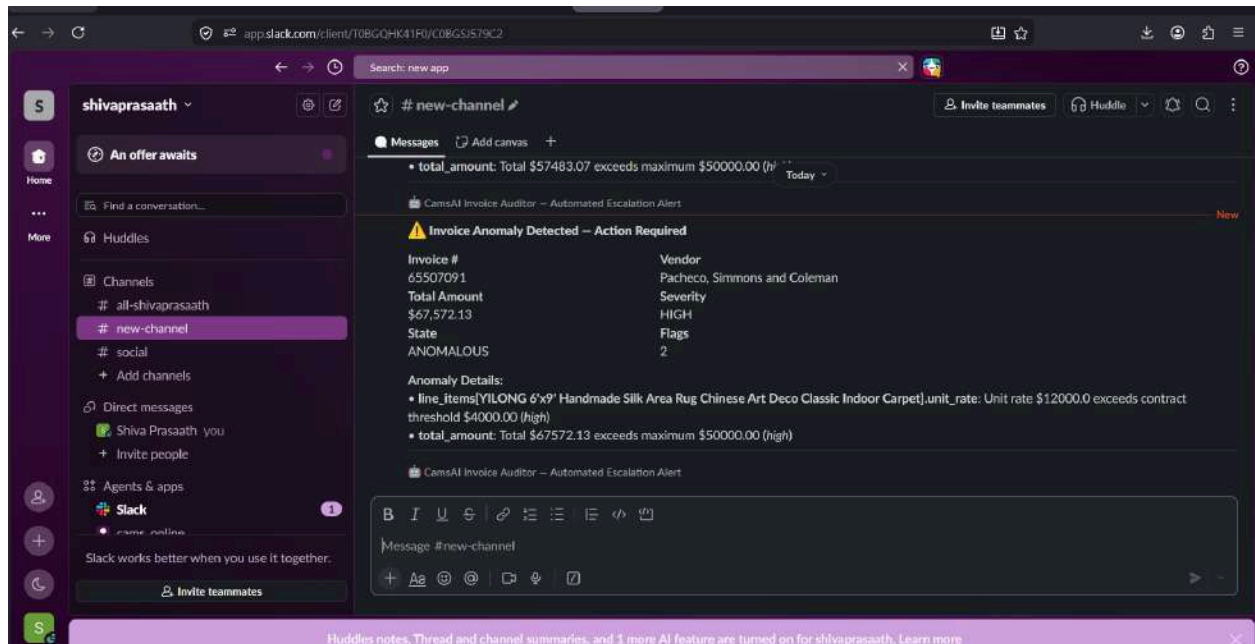
	Created	Expires at	Last used	Cost
	Jul 12, 2026	Jul 19, 2026	Jul 12, 2026	\$0.98

```
FO - Final State : OrchestratorState.COMPLETE
FO - Compliance : 100.0/100
FO - Flags : 0
FO - Ledger ID : 22c25286-0ff6-4022-ae1d-c932102c02bb

2026-07-12 09:05:05,409 [orchestrator] INFO -
2026-07-12 09:05:05,424 [orchestrator] INFO -
2026-07-12 09:05:05,424 [orchestrator] INFO - @ CSV MULTI LOG
2026-07-12 09:05:05,424 [orchestrator] INFO - File : /app/invoice_audit_log.csv
2026-07-12 09:05:05,425 [orchestrator] INFO - Rows : 1 Invoice(s) logged
2026-07-12 09:05:05,425 [orchestrator] INFO -
2026-07-12 09:05:05,929 [orchestrator] INFO -
2026-07-12 09:05:05,929 [orchestrator] INFO - SCENARIO : OCR - voxel51 sample #50 (tabular layout, currency parsing)
2026-07-12 09:05:05,929 [orchestrator] INFO - Invoice : #f:voxel51:50
2026-07-12 09:05:05,932 [orchestrator] INFO -
2026-07-12 09:05:05,932 [orchestrator] INFO - # START: HF:voxel51:50
2026-07-12 09:05:05,940 [orchestrator] INFO - [orchestrator] * INGEST | invoice-#f:voxel51:50
2026-07-12 09:05:05,941 [Agent1-Ingestion] INFO - [Agent1] OCR path: voxel51 sample #50
2026-07-12 09:05:05,941 [Agent1-Ingestion] INFO - [Agent1-ocr] voxel51 sample #50
2026-07-12 09:05:05,941 [Agent1-Ingestion] INFO - [12-07-2026 09:05:15] 127.0.0.1:39000 - "GET /api/v1/heartbeat HTTP/1.1" 200
2026-07-12 09:05:05,941 [Agent1-Ingestion] INFO - 127.0.0.1:37998 - "GET /health HTTP/1.1" 200 OK
```

As you can see, The orchestrator is running simultaneously and the current images processed and flagged are seen in the other tap and the api hit and cost details are mentioned in background.

Anomaly message triggered to SLACK :



Final ran data output

```

MINGW64:/c/Users/Asus
if not key or key in seen: continue
seen.add(key)
d = row[6] if len(row) > 6 else ''
if 'APPROVED' in d: approved += 1
elif 'FLAGGED' in d: flagged += 1
elif 'ERROR' in d: error += 1
print(f'Unique: {approved+flagged+error} | Approved: {approved} | Flagged: {flagged} | Error: {error}')
Unique: 108 | Approved: 85 | Flagged: 23 | Error: 0

asus@DESKTOP-CFFJGVE MINGW64 ~
$ python -c "
import csv
with open('C:/Cams_Test_F/camsai-invoice-auditor/invoice_audit_log.csv') as f:
    rows = list(csv.reader(f))
seen = set()
approved = flagged = error = 0
for row in rows[1:]:
    if not row: continue
    key = row[1] if len(row) > 1 else ''
    if not key or key in seen: continue
    seen.add(key)
    d = row[6] if len(row) > 6 else ''
    if 'APPROVED' in d: approved += 1
    elif 'FLAGGED' in d: flagged += 1
    elif 'ERROR' in d: error += 1
print(f'Unique: {approved+flagged+error} | Approved: {approved} | Flagged: {flagged} | Error: {error}')
Unique: 122 | Approved: 97 | Flagged: 25 | Error: 0

asus@DESKTOP-CFFJGVE MINGW64 ~
$

```

There are 122 unique images processed and 97 of them are approved and 25 are flagged as anomaly and 0 error triggered.

Please find the attached CSV which clearly shows the audit logs.

Thanks and regards,

Shiva Prasaath SB (+91 9443211197, shivabalamurugan@gmail.com)